

3. L'entité JPA (mapping objet-relationnel)

Étape 2 — EILMS : Database Persistence Layer (JDBC, PostgreSQL, JPA/Hibernate)

Fichier concerné : `src/main/java/com/eilms/entity/WarehouseItemEntity.java`

Pour que Hibernate puisse convertir automatiquement un objet Java en ligne de table SQL (et inversement), on annote la classe avec des métadonnées JPA.

```
@Entity
@Table(name = "warehouse_items")
public class WarehouseItemEntity {

    @Id
    @Column(name = "item_id", length = 50)
    private String id;

    @Column(name = "item_name", nullable = false, length = 150)
    private String name;

    @Column(name = "stock_quantity", nullable = false)
    private int quantity;

    // Constructeur sans argument requis par JPA (réflexion)
    public WarehouseItemEntity() {}

    public WarehouseItemEntity(String id, String name, int quantity) {
        this.id = id;
        this.name = name;
        this.quantity = quantity;
    }

    // Getters et Setters
    public String getId() { return id; }
    public void setId(String id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public int getQuantity() { return quantity; }
    public void setQuantity(int quantity) { this.quantity = quantity; }
}
```

Explication détaillée

@Entity : marque la classe comme une entité persistante gérée par JPA/Hibernate — chaque instance correspondra à une ligne dans une table de la base.

@Table(name = "warehouse_items") : associe explicitement cette classe à la table SQL « warehouse_items ». Sans cette annotation, Hibernate utiliserait par défaut le nom de la classe.

@Id : désigne le champ « id » comme clé primaire — c'est ce qui identifie chaque ligne de façon unique dans la table, exactement comme `equals()`/`hashCode()` le faisait pour l'objet en mémoire à l'étape 1.

@Column(name=..., nullable=..., length=...) : personnalise le mapping colonne par colonne. Exemple : « name » devient la colonne « item_name », de type `varchar(150)`, et ne peut pas être nulle (`nullable = false`).

Constructeur sans argument : obligatoire pour JPA. Quand Hibernate lit une ligne SQL, il utilise la réflexion Java pour créer d'abord un objet vide via ce constructeur, puis remplit ses champs — sans lui, Hibernate ne

peut pas instancier l'entité.

Analyse pédagogique du professeur

Annotations de métadonnées JPA : @Entity marque la classe comme entité persistante ; @Table(name = "warehouse_items") la mappe explicitement à la table relationnelle cible.

Spécification de la clé primaire : @Id désigne « id » comme colonne de clé primaire pour la résolution d'identité en base de données.

Contrainte du constructeur sans argument : les fournisseurs JPA utilisent la réflexion Java pour instancier dynamiquement les entités en convertissant les lignes de ResultSet SQL en objets Java. Un constructeur par défaut public ou protected est strictement requis.